

Introduction to Dart Programming Language

What is Dart?

Dart is a general-purpose, high-level modern programming language which is originally developed by Google. It is the new programming language that emerged in 2011, but its stable version was released in June 2017. Dart is not so popular at that time, but it gains popularity when it is used by the Flutter.

Dart is a dynamic, class-based, object-oriented programming language with closure and lexical scope. Syntactically, it is quite similar to Java, C, and JavaScript. If you know any of these programming languages, you can easily learn the Dart programming language.

Dart is an open-source programming language that is widely used to develop mobile applications, modern web applications, desktop applications, and the Internet of Things.

It is a compiled language and supports two types of compilation techniques.

- **AOT** (Ahead of Time) - It converts the Dart code into the optimized JavaScript code with the help of the dar2js compiler and runs on all modern web-browser. It compiles the code at build time.
- **JIT** (Just-In-Time) - It converts the byte code into the machine code (native code), but only code that is necessary.

History of Dart

Dart was revealed for the first time at the GOTO conference in the month of 10th - 12th October 2011 at Aarhus, Denmark. It is initially designed by Lars Bark and Kesper and developed by Google.

The first version 1.0 of Dart was released on November 14th, 2013, and intended as a replacement for JavaScript.

In July 2014, the first edition of Dart language was approved by Ecma International approved at its 107th General Assembly.

The first version was criticized due to a malfunction on the web and this plan was dropped in 2015 with the 1.9 release of Dart.

The second version of Dart 2.0 was released in August, including a sound-type system.

The recent version of Dart 3.0 is enhanced with the extension method, which enables us to add any type of functionality.

Why Dart?

- Dart is a platform-independent language and supports all operating systems such as Windows, Mac, Linux, etc.
- It is an open-source language, which means it is available free for everyone. It comes with a BSD license and is recognized by the ECMA standard.
- It is an object-oriented programming language and supports all features of oops such as inheritance, interfaces, and optional type features.
- Dart is very useful in building real-time applications because of its stability.
- Dart comes with the dar2js compiler which transmits the Dart code into JavaScript code that runs on all modern web browsers.
- The stand-alone Dart VM permits Dart code to run in a command-line interface environment.

Some of the characteristics of Dart Programming are as follows:

- Type inference
- Statically typed
- String Expressions
- Features of functional and object-oriented programming
- Null Safe
- Dart has the ability for developing a fast variety of mobile applications across different platforms.

Variables and Data Types

Variables

As in any programming language, variables are one of the basics and Dart comes with support for type inference. A variable is a placeholder in the computer's memory that holds some value. Variables are also referred to as identifiers.

Rules for variable declaration in Dart are as follows:

- A variable in Dart cannot comprise a special character such as whitespace, Unicode character, keywords, and mathematical symbols.
- The first character of a variable cannot be a digit. It can be uppercase or lowercase alphabet ([A-Z]/{a-z}). Variable names are case-sensitive.
- Variable names may contain underscore (_) and dollar (\$) sign but it does not contain special character such as @, #, &, ^ or *.

A typical example of the creation and initialization of a variable is as follows:

```
var value = 17;
var myName = "Ibrahim";
```

In the example above, the value is an integer while myName is a string. Like Java and C#, Dart is able to **infer** the type of the variable by looking at the value you've assigned. In other words, the Dart compiler is smart enough to figure out by itself which is the correct type of variable.

```
int value = 17;

String myName = "Ibrahim";
```

This code is identical to the preceding example with the only difference being that here the types have been typed explicitly. There would also be a third valid way to initialize variables, but you should almost never use it.

```
dynamic value = 17;

dynamic myName = "Ibrahim";
```

dynamic can be used with any type; any value can be assigned to it and the compiler won't complain. The type of a dynamic variable is evaluated at runtime and thus, for proper usage, you'd need to work with checks and type casts. According to the Dart guidelines you should:

1. Prefer initializing variables with **var** as much as you can;
2. When the type is not so easy to guess, initialize it explicitly to increase the readability of the code;
3. Use **Object** or **dynamic** only if it's really needed but it's almost never the case.

Actually, we could say that **dynamic** is not really a type: it's more of a way to turn off static analysis and tell the compiler you know what you're doing. The only case in which you'll deal with it will come in the Flutter part in regard to JSON encoding and decoding.

A variable declared as **final** can be set only once and if you try to change its content later, you'll get an error. For example, you won't be able to successfully compile this code:

```
final name = "Ibrahim";

name = "Ibrahim"; // 'name' is final and cannot be changed
```

You can also notice that the **final** can automatically infer the type exactly like **var** does. This keyword can be seen as a "restrictive var" as it deduces the type automatically but does not allow changes.

```
// Very popular - Automatic type deduction
final name = "Ibrahim";
// Generally unnecessary - With type annotation
final String nickName = "Bashir";
```

If you want you can also specify the type but it's not required. So far, we've only shown examples with strings, but of course, both **final** and **var** can be used with complex data types (classes, enums) or methods.